NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 4: Functions, Decomposition, and Abstraction

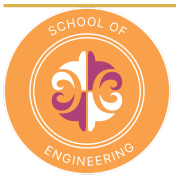
Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Identify a task ⟶ Define its interface ⟶ Implement and test it ⟶ Compose trusted pieces



About these notes

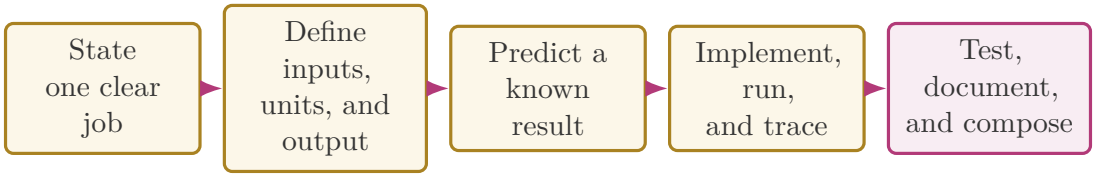
These reading notes expand the complete Week 4 lecture-slide set for *ENG 101 - Programming for Engineers*. The three lectures develop function definitions and calls, parameters and arguments, return values, scope, documentation, testing, decomposition, abstraction, and the construction of readable engineering workflows. The organization, terminology, examples, cautions, classroom activities, and engineering interpretation follow the Week 4 slides. The explanations are expanded so that the document can be read independently as preparation, review, and reference material.

Every executable Python sample includes comments. The comments identify the purpose of a function, the physical meaning and units of its parameters, the value it returns, the assumptions that must hold, the reason a test is useful, and the source of deliberately introduced defects. This convention is not only for beginners: readable comments and clear interfaces help engineering teams audit calculations and reuse code safely.

Week 4 marks a major transition. Earlier programs were mostly sequences of statements. We now package meaningful tasks into named, reusable components. A trustworthy function is a small engineering promise: its name states the job, its parameters expose the information required, its return value carries the result, and its documentation and tests provide evidence for use.

How to use these notes

Functions become understandable when each one is treated as a small engineering experiment. The recommended Week 4 study cycle is:



For every function in these notes:

1. read the function name aloud and state the job in one sentence;
2. identify each parameter, its expected type, physical meaning, and unit;
3. distinguish the parameter names in the definition from the argument values in a call;
4. predict the returned value before running the code;
5. trace control into the function body and back to the calling statement;
6. check whether the function prints information, returns a value, or does both;
7. test a simple case that can be checked independently by hand;
8. inspect boundary and invalid cases that matter to the function contract;
9. explain what assumptions make the result meaningful;
10. decide whether the function has one focused responsibility and can be composed with other functions.

Engineering check

A function can be syntactically correct and numerically active while still being unsafe to reuse. Wrong argument order, hidden global dependencies, undocumented units, early return statements, or unsuitable assumptions may produce plausible numbers. The caller remains responsible for understanding the interface and applying the result within the stated model.

Week 4 structure

Lecture	Main theme	Core ideas
Lecture 10	Defining and calling functions	Why repeated work should become a function; built-in and user-defined functions; anatomy of a definition; calls; parameters and arguments; positional and keyword arguments; returned values; `print()` versus `return`; composition; first tests and debugging.
Lecture 11	Interfaces and trustworthy functions	Function interfaces and contracts; positional, keyword, and default arguments; local scope; call frames; explicit inputs; multiple returned values; decisions and loops in functions; docstrings; optional type hints; assertions; representative, boundary, and tolerance-based tests.
Lecture 12	Decomposition and abstraction	Dividing a larger engineering task into focused functions; data flow; top-down design and bottom-up testing; abstraction through stable interfaces; separation of input, calculation, decision, and reporting; refactoring bisection; bar-design workflow; small function libraries; Assignment 2 preparation.

Week 4 learning outcomes

By the end of Week 4, you should be able to:

1. explain why meaningful repeated work should be packaged as a function;
2. define a function using `def` and identify its name, parameters, body, and return statement;
3. distinguish a function definition from a function call;
4. distinguish parameters from arguments and trace parameter binding;
5. use positional and keyword arguments appropriately;
6. explain why argument order, units, and assumptions remain the caller's responsibility;
7. distinguish printed output from a reusable returned value;
8. explain why a function without an executed value-returning statement returns `None`;
9. use a returned value in later expressions and function calls;
10. design a function interface before writing its body;

11. use default parameters without treating a default policy as a universal engineering truth;
12. explain local scope and why local names from different calls do not conflict;
13. pass required information explicitly rather than relying on hidden global variables;
14. return two related results and unpack them at the call site;
15. write functions that contain decisions or loops while preserving one clear responsibility;
16. write a docstring that records purpose, input meaning and units, output meaning and units, and assumptions;
17. interpret a function specification as a contract between caller and implementation;
18. use assertions and tolerance-based comparisons to test returned values;
19. test representative, boundary, and invalid cases;
20. diagnose missing calls, missing returns, wrong argument order, scope errors, hidden dependencies, and premature returns;
21. decompose a larger engineering problem into smaller tasks with clear data flow;
22. explain abstraction as using a documented interface without repeating every implementation detail;
23. separate input, calculation, decision, and presentation responsibilities;
24. refactor a numerical method into a reusable function with a precise contract;
25. build, test, and explain a small function-based engineering workflow.

Key idea

The central Week 4 message is: functions are not merely syntax. They organize engineering thought into named, testable, reusable promises. Decomposition creates structure; abstraction lets us use that structure without carrying every internal statement in our heads.

Contents

How to use these notes 3
- Week 4 structure . . . 4
- Week 4 learning outcomes . . . 4

1 Lecture 10: Defining and Calling Functions 9
- 1.1 From repeated statements to a reusable idea . . . 9
- 1.2 A function as input–process–output . . . 10
- 1.3 Built-in functions have already introduced the pattern . . . 11
- 1.4 Your first engineering function . . . 11
- 1.5 Anatomy of a function definition . . . 12
- 1.6 Definition and call are different events . . . 13
- 1.7 What happens during a function call? . . . 13
- 1.8 Parameters and arguments . . . 14
- 1.9 Positional arguments: order carries meaning . . . 14
- 1.10 Keyword arguments make meaning visible . . . 15
- 1.11 Returned values support later calculations . . . 15
- 1.12 `print()` and `return` answer different needs . . . 16
  - 1.12.1 The `None` surprise . . . 16
- 1.13 Compose small functions into a calculation chain . . . 17
- 1.14 Engineering formulas make natural functions . . . 17
- 1.15 A function call is an expression . . . 18
- 1.16 Design the interface before the body . . . 18
- 1.17 Live-coding ladder: grow one function carefully . . . 19
- 1.18 Common first-function syntax errors . . . 20
  - 1.18.1 Missing colon . . . 20
  - 1.18.2 Missing indentation . . . 20
  - 1.18.3 Missing call parentheses . . . 20
- 1.19 Debugging clinic: local result and missing call . . . 21
- 1.20 First test: use numbers you can verify by hand . . . 21
- 1.21 Lecture 10 summary . . . 22
- 1.22 Lecture 10 self-check . . . 22
  - 1.22.1 Density exit ticket . . . 22
  - 1.22.2 Explain the call . . . 23

2 Lecture 11: Function Interfaces, Scope, Documentation, and Tests 24
- 2.1 Retrieval warm-up: trace every printed line . . . 24
- 2.2 The interface is the visible agreement with the caller . . . 25
- 2.3 Three ways to supply arguments . . . 25
  - 2.3.1 Positional arguments . . . 25
  - 2.3.2 Keyword arguments . . . 26
  - 2.3.3 Default arguments . . . 26
- 2.4 Keyword arguments clarify long engineering calls . . . 26
- 2.5 Scope answers: where does a name exist? . . . 27
  - 2.5.1 Local variables stay inside their function . . . 27
  - 2.5.2 Caller names and parameter names may differ . . . 28
  - 2.5.3 Two functions may safely use the same local name . . . 28
- 2.6 Prefer explicit inputs to hidden dependencies . . . 29
- 2.7 Returning two related results . . . 29

2.8 Functions can contain engineering decisions . . . 30
2.9 Functions can contain loops without exposing the loop . . . 30
2.10 A docstring records engineering meaning . . . 31
2.11 A specification is a contract . . . 32
2.11.1 Caller assumptions . . . 32
2.11.2 Function guarantee . . . 32
2.12 Optional type hints can strengthen readability . . . 32
2.13 Assertions make expectations executable . . . 33
2.14 Test more than one convenient example . . . 33
2.15 Floating-point functions need tolerance-based tests . . . 34
2.16 Common function failures and their clues . . . 35
2.16.1 Premature return example . . . 35
2.17 Debugging activity: four distinct problems . . . 36
2.18 Function interface checklist . . . 37
2.19 Lecture 11 summary . . . 37
2.20 Lecture 11 self-check . . . 37
2.20.1 Design a density contract . . . 37
2.20.2 Boundary reasoning . . . 38

3 Lecture 12: Decomposition and Abstraction in Engineering Programs 39
3.1 Retrieval warm-up: draw the function boundaries . . . 39
3.2 Decomposition: turn one large task into smaller questions . . . 40
3.3 The guiding question: what is the job of this function? . . . 41
3.4 A decomposition table comes before code . . . 41
3.5 Data flow connects the components . . . 42
3.6 Top-down design and bottom-up confidence . . . 42
3.6.1 Top-down: plan the system . . . 42
3.6.2 Bottom-up: build trust . . . 42
3.7 A well-decomposed program reads like an engineering procedure . . . 43
3.8 Abstraction: use the interface, hide repeated detail . . . 44
3.9 Implementation can change while the interface remains stable . . . 44
3.10 Good abstraction depends on good names . . . 44
3.11 Separate computation from input and reporting . . . 45
3.12 Refactor Week 3: package bisection as a reusable function . . . 46
3.12.1 Reuse the numerical method . . . 46
3.13 Specify the numerical abstraction precisely . . . 47
3.14 Bar-design functions: calculations and decision . . . 48
3.15 The integrated calculation function . . . 48
3.16 Keep reporting separate from calculation . . . 49
3.17 Test pieces before testing the whole program . . . 50
3.18 A small engineering library collects reusable functions . . . 51
3.19 Activity: decompose an energy-cost calculator . . . 51
3.20 Pair design review: can another engineer use the function? . . . 52
3.21 Assignment 2: function-based problem solving . . . 53
3.22 Week 4 scope boundary . . . 53
3.23 Lecture 12 summary . . . 54
3.24 Lecture 12 self-check . . . 54
3.24.1 Decompose a projectile calculation . . . 54
3.24.2 Explain abstraction . . . 55

4 Week 4 Synthesis and Practice 56
4.1 The Week 4 concept map . . . 56

4.2 Week 4 quick reference . . . 56
4.2.1 Definitions, calls, and return values . . . 56
4.2.2 Engineering design habits . . . 57
4.3 Glossary . . . 57
4.4 Reading function code systematically . . . 59
4.5 Cumulative practice: definitions, calls, and return values . . . 59
4.5.1 Practice 1: trace a function call . . . 59
4.5.2 Practice 2: definition without a call . . . 60
4.5.3 Practice 3: print versus return . . . 60
4.6 Cumulative practice: parameters, scope, and contracts . . . 61
4.6.1 Practice 4: repair a hidden dependency . . . 61
4.6.2 Practice 5: write a contract . . . 61
4.6.3 Practice 6: local names . . . 62
4.7 Cumulative practice: testing . . . 62
4.7.1 Practice 7: representative and boundary tests . . . 62
4.7.2 Practice 8: tolerance-based test . . . 63
4.7.3 Practice 9: find the premature return . . . 63
4.8 Cumulative practice: decomposition and abstraction . . . 64
4.8.1 Practice 10: decompose a resistor report . . . 64
4.8.2 Practice 11: identify interfaces and data flow . . . 65
4.8.3 Practice 12: refactor repeated conversions . . . 66
4.9 Integrated practice: function-based bar design . . . 66
4.10 Readiness checklist for Week 5 . . . 68

Sources and Suggested Reading **70**

1 Lecture 10: Defining and Calling Functions

Lecture-slide alignment: Lecture 10 of the Week 4 slides: repeated engineering calculations; function anatomy; definitions and calls; parameters and arguments; positional and keyword arguments; returned values; `print()` versus `return`; composition; formula functions; interface design; syntax repair; scope diagnosis; and first tests.

Learning outcomes

By the end of this chapter, you should be able to:

1. explain why meaningful repeated work should become a function;
2. define a function using `def`;
3. identify a function's name, parameters, colon, indented body, and return statement;
4. distinguish defining a function from calling it;
5. distinguish parameters from arguments;
6. use positional and keyword arguments;
7. store and reuse a returned value;
8. distinguish communication with `print()` from computation with `return`;
9. compose small functions into a readable calculation chain;
10. diagnose common first-function errors;
11. verify a function using a hand-checkable case.

1.1 From repeated statements to a reusable idea

Consider three stress calculations:

```
# Store force and area for the first loading case.
force_1 = 5000       # N
area_1 = 0.002       # m^2
stress_1 = force_1 / area_1  # Pa

# Store force and area for the second loading case.
force_2 = 8000       # N
area_2 = 0.003       # m^2
stress_2 = force_2 / area_2  # Pa

# Store force and area for the third loading case.
force_3 = 12000      # N
area_3 = 0.004       # m^2
stress_3 = force_3 / area_3  # Pa
```

Code 1.1: Repeated stress calculations before functions.

The formula is repeated:

$$\sigma = \frac{F}{A}.$$

The force and area change, but the engineering operation remains the same. This suggests a reusable tool with:

- a meaningful name, such as `calculate_stress`;
- two inputs: force and area;
- one process: divide force by area;
- one output: stress.

Definition

A **function** is a named, reusable unit of computation. It may receive input values through parameters, execute an indented body, and return a result to the calling code.

Functions are useful for more than shortening code. They make the underlying engineering ideas visible. A long script may convert units, calculate stress, calculate a factor of safety, classify a design, and print a report. If all of those ideas are hidden in one uninterrupted sequence, it becomes difficult to:

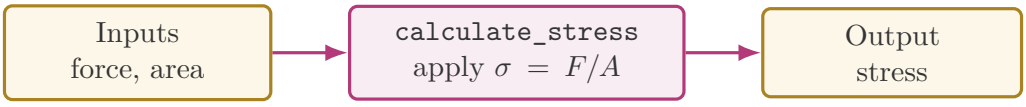
- evaluate many designs without copying code;
- change one formula without disturbing unrelated statements;
- test the unit conversion separately;
- reuse only the stress calculation in another program;
- locate the source of an incorrect result.

Design principle

Give each meaningful task a name and a clear interface. A good first question is: *Can this task be described as one useful operation with identifiable inputs and an identifiable output?*

1.2 A function as input–process–output

The stress function can be pictured as:



The caller should be able to understand the tool through its name and interface. The internal statements implement the operation. This separation becomes increasingly valuable as programs grow.

1.3 Built-in functions have already introduced the pattern

You have already called many functions:

```
# Display a text value for a human reader.
print("ENG 101")

# Return the number of characters in a string.
length = len("sensor")

# Return the nonnegative magnitude of a number.
magnitude = abs(-12.5)

# Return the largest of the supplied values.
largest = max(3, 8, 5)

# Return the type of a numerical object.
kind = type(3.14)
```

Code 1.2: Examples of built-in Python functions.

Each call contains:

- a function name;
- parentheses;
- zero or more arguments inside the parentheses;
- defined behavior;
- sometimes a value returned to the caller.

The next step is to define our own named tools.

1.4 Your first engineering function

```
# Define a reusable calculation for uniform axial stress.
def calculate_stress(force, area):
    # Apply sigma = F/A using force in N and area in m^2.
    stress = force / area

    # Send the stress in Pa back to the caller.
    return stress

# Call the function with one force-area pair.
result = calculate_stress(5000, 0.002)

# Convert Pa to MPa only for the displayed report.
print(f"Stress = {result / 1e6:.2f} MPa")
```

Code 1.3: A first function for uniform axial stress.

The output is:

```
Stress = 2.50 MPa
```

Code 1.4: Output from the first stress function.

The units follow from the inputs:

$$\frac{\mathrm{N}}{\mathrm{m}^2} = \mathrm{Pa}.$$

Thus the function returns pascals. The caller converts the result to megapascals for presentation.

Engineering check

The function does not know that the numbers represent newtons and square metres. The unit interpretation comes from the interface, names, documentation, and disciplined use by the caller.

1.5 Anatomy of a function definition

Study the header and body:

```
# "def" begins a function definition.
def calculate_stress(force, area):
    # This indented statement belongs to the function body.
    stress = force / area

    # "return" sends a result to the caller and ends this call.
    return stress
```

Code 1.5: The parts of a function definition.

The parts are:

`def`
: The Python keyword that begins a function definition.

`calculate_stress`
: A meaningful function name written in lowercase `snake_case`.

`force, area`
: Parameter names used inside the function.

Colon
: The `:` marks the beginning of the function body.

Indented body
: Every statement belonging to the function must be indented consistently.

`return stress`
: The return statement sends the calculated result to the caller.

A function body is defined by indentation. A statement accidentally moved to the left may execute outside the function. A statement indented inconsistently may cause an `IndentationError` or change the intended control flow.

1.6 Definition and call are different events

A definition creates the tool:

```
# Define how current should be calculated from voltage and resistance.
def calculate_current(voltage, resistance):
    # Apply Ohm's law I = V/R and return the current.
    return voltage / resistance
```

Code 1.6: Defining a current calculation.

Python records the definition. The body is not executed merely because the definition exists.

A call uses the tool:

```
# Call the function with 12 V and 4 ohm.
current = calculate_current(12, 4)

# Display the returned current.
print(current)
```

Code 1.7: Calling the previously defined function.

This prints:

```
3.0
```

Code 1.8: Returned current.

Key idea

A function can be defined perfectly and still appear to do nothing because it was never called. Definition tells Python what the tool is; a call asks Python to use it.

1.7 What happens during a function call?

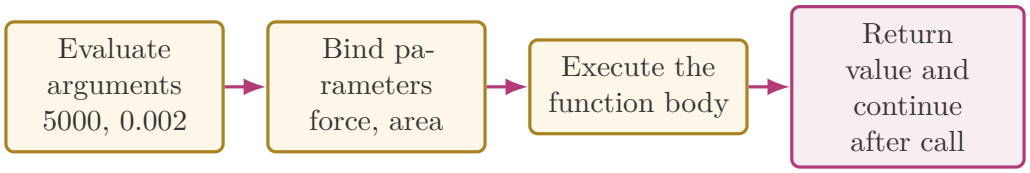
For the call

```
# Use 5000 N and 0.002 m^2 as arguments.
result = calculate_stress(5000, 0.002)
```

Code 1.9: A call whose execution will be traced.

Python performs four conceptual stages:

1. **Evaluate the arguments.** The expressions `5000` and `0.002` are evaluated.
2. **Bind the parameters.** Inside this call, `force` refers to 5000 and `area` refers to 0.002.
3. **Execute the body.** Python calculates `stress = force / area`.
4. **Return and continue.** The value is sent back, assigned to `result`, and execution continues after the call.



1.8 Parameters and arguments

```
# The names force and area are parameters.
def calculate_stress(force, area):
    # Return the value represented by the parameters in this call.
    return force / area

# The values 5000 and 0.002 are arguments.
stress = calculate_stress(5000, 0.002)
```

Code 1.10: Parameters in a definition and arguments in a call.

Definition

A **parameter** is a local name written in the function definition. An **argument** is a value or expression supplied by the caller during a function call.

Parameters describe what kinds of information the function needs. Arguments provide the actual information for one use.

1.9 Positional arguments: order carries meaning

In a positional call, the first argument is bound to the first parameter, the second argument to the second parameter, and so on.

```
# Bind force=5000 N and area=0.002 m^2 in the intended order.
stress = calculate_stress(5000, 0.002)
```

Code 1.11: Correct positional argument order.

Reversing the values is syntactically legal:

```
# BUG: 0.002 is incorrectly treated as force and 5000 as area.
stress = calculate_stress(0.002, 5000)
```

Code 1.12: A syntactically valid but physically wrong call.

Python can divide the two numbers, but the engineering meaning is wrong. The returned number is not the intended stress.

Engineering check

A syntactically valid call can be physically meaningless. The caller remains responsible for parameter order, input units, valid ranges, and model assumptions.

1.10 Keyword arguments make meaning visible

Keyword arguments name the parameter at the call site:

```
# Name both arguments so their engineering roles are visible.
stress = calculate_stress(
    force=5000,   # N
    area=0.002    # m^2
)
```

Code 1.13: Calling a function with keyword arguments.

Advantages include:

- the meaning of each number is visible;
- the order becomes less ambiguous;
- long engineering calls are easier to review;
- parameter names can help communicate expected quantities.

Keyword arguments do not automatically guarantee correct units. A parameter called `area` remains ambiguous unless documentation or a unit-bearing name clarifies whether the value is in m^2, mm^2, or another unit.

1.11 Returned values support later calculations

```
# Define current from voltage and resistance.
def calculate_current(voltage, resistance):
    # Return current in A when voltage is in V and resistance in ohm.
    return voltage / resistance

# Calculate current for a 12 V supply and 4 ohm resistance.
current = calculate_current(12, 4)

# Reuse the current to calculate electrical power in W.
power = 12 * current

# Calculate energy in J for 3600 s of operation.
energy = power * 3600

# Communicate the calculated values.
print(f"Current = {current:.2f} A")
print(f"Energy = {energy:.0f} J")
```

Code 1.14: Reusing a returned value in later calculations.

A returned value can be:

- stored in a variable;
- printed;
- used in another expression;

- passed as an argument to another function.

This is why calculation functions normally return results rather than only printing them.

1.12 `print()` and `return` answer different needs

Question	`print()`	`return`
Primary purpose	Communicate information to a human	Send a value back to the caller
Reusable in a calculation?	Not by itself	Yes
Ends the function call?	No	Yes, when executed
Typical location	Reporting or user-interface layer	Calculation or decision function

A clear pattern is:

```
# Define a reusable electrical-power calculation.
def calculate_power(voltage, current):
    # Return power in W when voltage is in V and current in A.
    return voltage * current

# Store the value returned by the function.
power = calculate_power(12, 3)

# Format and display the result for a human reader.
print(f"Power = {power:.1f} W")
```

Code 1.15: Return the calculation, then print it outside the function.

1.12.1 The `None` surprise

Consider this faulty design:

```
# Define a function that calculates and displays stress.
def calculate_stress_bad(force, area):
    # Calculate stress in Pa.
    stress = force / area

    # Displaying the value does not send it back to the caller.
    print(stress)

# The function has no value-returning statement, so value becomes None.
value = calculate_stress_bad(5000, 0.002)

# Show what was actually stored by the caller.
print("Stored value:", value)
```

Code 1.16: A function that prints but does not return.

The output is:

```
2500000.0
Stored value: None
```

Code 1.17: Printing inside the function does not create a returned value.

A function without an executed value-returning statement returns `None`. The printed number was visible to the human, but it was not available to the rest of the program as the function's result.

Seeing a number on the screen does not prove that a function returned that number. Store the call in a variable and inspect the value when you are unsure.

1.13 Compose small functions into a calculation chain

```python
# Calculate current from voltage and resistance.
def calculate_current(voltage, resistance):
    # Return current in A for inputs in V and ohm.
    return voltage / resistance

# Calculate power from voltage and current.
def calculate_power(voltage, current):
    # Return power in W for inputs in V and A.
    return voltage * current

# First obtain current from the circuit inputs.
current = calculate_current(12, 4)

# Pass the returned current into the next function.
power = calculate_power(12, current)

# Report the two engineering quantities.
print(f"Current = {current:.2f} A")
print(f"Power = {power:.2f} W")
```

Code 1.18: Composing current and power functions.

The main workflow reads like the engineering procedure:

$$\text{voltage, resistance} \longrightarrow \text{current} \longrightarrow \text{power}.$$

The arithmetic details remain inside the functions. The calling code communicates the higher-level sequence.

1.14 Engineering formulas make natural functions

Many elementary relationships have clear inputs and outputs.

```python
# Return uniform axial stress in Pa from force in N and area in m^2.
def calculate_stress(force, area):
    return force / area
```

```
4
5  # Return dimensionless engineering strain from two lengths in the same unit.
6  def calculate_strain(delta_length, original_length):
7      return delta_length / original_length
8
9  # Return kinetic energy in J from mass in kg and velocity in m/s.
10 def calculate_kinetic_energy(mass, velocity):
11     return 0.5 * mass * velocity**2
12
13 # Return density in kg/m^3 from mass in kg and volume in m^3.
14 def calculate_density(mass, volume):
15     return mass / volume
```

Code 1.19: A small set of formula functions.

A useful function-selection question is:

> Can the task be described as one meaningful calculation with clear inputs and a clear output?

Not every individual arithmetic symbol needs its own function. Function boundaries should reflect useful ideas, not merely make the program longer.

1.15 A function call is an expression

Because a function call evaluates to its returned value, it can appear wherever a compatible value can appear.

```
1  # Return average velocity in m/s from distance in m and time in s.
2  def calculate_velocity(distance, time):
3      return distance / time
4
5  # Store the returned value directly.
6  speed_m_s = calculate_velocity(100, 9.58)
7
8  # Use another call as part of a unit-conversion expression.
9  speed_km_h = calculate_velocity(100, 9.58) * 3.6
10
11 # Display both representations of the same speed.
12 print(speed_m_s)
13 print(speed_km_h)
```

Code 1.20: Using a function call as part of a larger expression.

Although direct expressions are legal, important intermediate quantities should often be stored using meaningful names. Doing so keeps units and engineering meaning visible.

1.16 Design the interface before the body

Suppose a device with power P operates for time t, and the required output is energy in joules:

$$E = Pt.$$

Before writing the arithmetic, specify:

1. a meaningful function name;
2. parameter names that reveal quantities and units;
3. the returned quantity and unit;
4. one call whose answer can be checked by hand.

One possible interface is:

```
# Planned interface:
# - power_w is electrical or mechanical power in W.
# - time_s is operating time in s.
# - the returned value will be energy in J.
def calculate_energy_j(power_w, time_s):
    # Apply E = P*t because 1 W = 1 J/s.
    return power_w * time_s

# Hand check: 10 W for 2 s should produce 20 J.
test_energy_j = calculate_energy_j(10, 2)
print(test_energy_j)
```

Code 1.21: Designing an interface before implementation.

The interface can be reviewed before the function body exists. This prevents unclear names and hidden unit assumptions from becoming embedded in the implementation.

1.17 Live-coding ladder: grow one function carefully

A useful classroom workflow is to add one concept at a time.

```
# Convert pressure from kPa to Pa.
def convert_kpa_to_pa(pressure_kpa):
    # One kilopascal equals 1000 pascals.
    return pressure_kpa * 1000
```

Code 1.22: Stage 1: define the pressure conversion.

```
# Convert a pressure of 250 kPa.
pressure_pa = convert_kpa_to_pa(250)
```

Code 1.23: Stage 2: call and store the function result.

```
# Display the returned pressure in Pa.
print(f"Pressure = {pressure_pa:.0f} Pa")
```

Code 1.24: Stage 3: communicate the converted value.

Then modify the call rather than the function body:

```
# Convert two different pressures without duplicating the formula.
pressure_85_pa = convert_kpa_to_pa(85)
pressure_250_pa = convert_kpa_to_pa(250)

# Report both converted values.
```

```
print(f"85 kPa = {pressure_85_pa:.0f} Pa")
print(f"250 kPa = {pressure_250_pa:.0f} Pa")
```

Code 1.25: Stage 4: reuse the function for two values.

1.18 Common first-function syntax errors

1.18.1 Missing colon

```
# BUG: the function header must end with a colon.
def stress(f, a)
    # This body cannot be parsed until the header is repaired.
    return f / a
```

Code 1.26: Faulty definition with a missing colon.

Correct header:

```
# End the definition header with a colon.
def stress(f, a):
    # Return the quotient after the syntax is valid.
    return f / a
```

Code 1.27: Corrected function header.

1.18.2 Missing indentation

```
# Define the function header correctly.
def stress(f, a):
# BUG: this return statement should be indented inside the function.
return f / a
```

Code 1.28: Faulty definition with an unindented body.

1.18.3 Missing call parentheses

```
# BUG: function arguments must be placed inside parentheses.
value = stress 5000, 0.002
```

Code 1.29: Faulty call with missing parentheses.

The repair strategy is systematic:

1. inspect the function header;
2. check the colon;
3. check consistent indentation;
4. check parentheses around arguments;
5. check the number of arguments;

6. check the exact function name;
7. repair one issue and run again.

1.19 Debugging clinic: local result and missing call

The following script fails:

```
# Define a power calculation.
def calculate_power(voltage, current):
    # Create a local variable inside the function.
    power = voltage * current

    # Return the local result to a future caller.
    return power

# BUG: no function call has created an outside variable named power.
print(power)
```

Code 1.30: A function that is defined but not called before an invalid external reference.

There are two related facts:

- the function was never called;
- `power` is local to the function body and is not directly available outside it.

A repair is:

```
# Define the reusable calculation.
def calculate_power(voltage, current):
    # Return power in W.
    return voltage * current

# Call the function and bind the returned value to an outside name.
power = calculate_power(12, 3)

# Display the outside variable.
print(power)
```

Code 1.31: Call the function and store the returned value.

Scope is developed in greater detail in Lecture 11.

1.20 First test: use numbers you can verify by hand

A small known case provides independent evidence:

```
# Define the stress function to be tested.
def calculate_stress(force, area):
    # Return uniform stress in Pa.
    return force / area

# For 10 N over 2 m^2, the hand-calculated answer is 5 Pa.
```

```
expected = 5

# Evaluate the function using the known case.
actual = calculate_stress(10, 2)

# Display the expected and actual values and their equality.
print("Expected:", expected)
print("Actual:", actual)
print("Pass:", actual == expected)
```

Code 1.32: Manual expected-versus-actual function test.

Simple cases are powerful because:

- the expected answer is independent of the code;
- mistakes are easier to locate;
- the component can be tested separately from a larger program;
- the result is easy to explain to another person.

One successful case is not a complete proof. It is the first piece of evidence. Later, test representative inputs, important boundaries, invalid values, and combinations likely to expose defects.

1.21 Lecture 10 summary

Section summary

A function packages one meaningful task behind a name. It receives arguments through parameters, executes an indented body, and can return a value that the rest of the program reuses. Defining and calling are different events. Positional argument order carries meaning; keyword arguments can make calls clearer. `print()` communicates, whereas `return` supplies a reusable result. Small formula functions can be composed into readable workflows and tested with hand-checkable cases.

1.22 Lecture 10 self-check

1.22.1 Density exit ticket

For $\rho = m/V$, write a function header, return statement, call for $m = 2.7\,\text{kg}$ and $V = 0.001\,\text{m}^3$, and the expected result.

```
# Return density in kg/m^3 from mass in kg and volume in m^3.
def calculate_density(mass_kg, volume_m3):
    # Apply rho = m/V.
```

```
    return mass_kg / volume_m3

# Evaluate the specified component.
density_kg_m3 = calculate_density(2.7, 0.001)

# The hand-calculated result is 2700 kg/m^3.
print(f"Density = {density_kg_m3:.1f} kg/m^3")
```

Code 1.33: Annotated density function and call.

1.22.2 Explain the call

For `calculate_stress(force=5000, area=0.002)`, identify:

1. the function name;
2. the parameters;
3. the arguments;
4. the expected returned quantity and unit;
5. one assumption that is not enforced by Python.

The name is `calculate_stress`. The parameters in the definition are `force` and `area`; the keyword arguments in this call are 5000 and 0.002. If they mean newtons and square metres, the returned quantity is uniform stress in pascals. An unenforced assumption is that the load is axial and the area is positive and expressed in square metres.

2 Lecture 11: Function Interfaces, Scope, Documentation, and Tests

Lecture-slide alignment: Lecture 11 of the Week 4 slides: interfaces as agreements; positional, keyword, and default arguments; local scope and call frames; explicit inputs; multiple returned values; functions containing decisions and loops; docstrings; specifications; optional type hints; assertions; representative and boundary tests; floating-point tolerances; debugging clues; and the interface checklist.

Learning outcomes

By the end of this chapter, you should be able to:

1. describe a function interface using its name, parameters, and returned value;
2. use positional, keyword, and default arguments appropriately;
3. explain that parameters and variables assigned inside a function are normally local;
4. trace values between caller scope and a temporary function scope;
5. pass required engineering quantities explicitly rather than hiding dependencies;
6. return two related results when this improves the interface;
7. write a function containing a decision or loop;
8. write a docstring that records purpose, units, output, and assumptions;
9. interpret a specification as a contract;
10. use assertions and tolerances to test returned values;
11. diagnose wrong argument order, missing returns, scope errors, hidden dependencies, and premature returns.

2.1 Retrieval warm-up: trace every printed line

```python
# Define a reusable power calculation.
def calculate_power(voltage, current):
    # Create a local result for this function call.
    power = voltage * current

    # Display an intermediate value while control is inside the function.
    print("Inside:", power)

    # Send the calculated power back to the caller.
    return power

# Calling the function moves execution into its body.
result = calculate_power(12, 3)

# After the return, execution continues here with result=36.
print("Outside:", result)
```

Code 2.1: Tracing output inside and outside a function.

The printed order is:

```
Inside: 36
Outside: 36
```

Code 2.2: Output order for the warm-up trace.

Control first reaches the call, enters the function, prints the inside value, returns 36, stores it in `result`, and finally prints the outside value. The local name `power` is not automatically available after the call; the caller receives only the returned value.

2.2 The interface is the visible agreement with the caller

A function interface should answer four questions:

1. What must the caller provide?
2. What does the function promise to return?
3. Which units and valid ranges are required?
4. Which model assumptions make the result meaningful?

For example:

```
# The caller supplies force and area.
def calculate_stress(force, area):
    # The function promises to return force divided by area.
    return force / area
```

Code 2.3: A minimal stress interface.

The code alone leaves some questions unanswered. Are the units newtons and square metres? Must area be positive? Is the result uniform axial stress? Those meanings belong in parameter names, documentation, and the function contract.

Function contract

A function contract describes the assumptions the caller must satisfy and the guarantee the function provides when those assumptions hold. The body may change, but the documented promise should remain stable unless the interface is intentionally revised.

2.3 Three ways to supply arguments

2.3.1 Positional arguments

```
# The first value binds to force and the second to area.
stress = calculate_stress(5000, 0.002)
```

Code 2.4: Compact positional call.

Positional calls are compact, but order carries meaning. They are most readable when the call is short and the parameter order is obvious.

2.3.2 Keyword arguments

```
# Name each argument so its role is visible at the call site.
stress = calculate_stress(
    force=5000,  # N
    area=0.002   # m^2
)
```

Code 2.5: Explicit keyword call.

Keyword calls help a reviewer inspect the physical meaning of each value without memorizing order.

2.3.3 Default arguments

A default value provides a common value when the caller does not supply one.

```
# Define a decision function with a documented default requirement.
def classify_design(factor_of_safety, required_fos=2.0):
    # Accept the design when the calculated factor meets the stated requirement.
    if factor_of_safety >= required_fos:
        return "acceptable"

    # This line handles every lower factor of safety.
    return "not acceptable"
```

Code 2.6: A default factor-of-safety requirement.

The call

```
# Use the default required factor of safety, 2.0.
print(classify_design(2.5))

# Override the default with a stricter requirement of 3.0.
print(classify_design(2.5, required_fos=3.0))
```

Code 2.7: Using and overriding the default argument.

produces:

```
acceptable
not acceptable
```

Code 2.8: Default and overridden classifications.

Engineering check

A default value expresses a normal software policy, not a universal engineering truth. The actual design code, standard, project requirement, or instructor specification must take priority. Defaults should be visible, documented, and easy to override.

2.4 Keyword arguments clarify long engineering calls

The Reynolds number is

$$\mathrm{Re} = \frac{\rho v D}{\mu}.$$

A positional call with four similar numbers is difficult to audit. Keyword arguments make the role of each value visible.

```
# Return Reynolds number from SI quantities.
def calculate_reynolds_number(
        density, velocity, diameter, viscosity):
    # Apply Re = rho*v*D/mu; the result is dimensionless.
    return density * velocity * diameter / viscosity

# Name each supplied quantity for readability.
re = calculate_reynolds_number(
    density=1000,      # kg/m^3
    velocity=2.0,      # m/s
    diameter=0.05,     # m
    viscosity=0.001    # Pa*s
)

# Report the dimensionless result.
print(f"Reynolds number = {re:.0f}")
```

Code 2.9: A Reynolds-number function and explicit keyword call.

This is easier to review, but the call still relies on documentation for units. Parameter names such as `density_kg_m3` could make units even more explicit, though names should remain readable.

2.5 Scope answers: where does a name exist?

Scope describes the region of a program in which a name can be used.

Parameters and variables assigned inside a function normally have **local scope**. They belong to that function call.

Key idea

Think of a function call as receiving a temporary workbench. The workbench contains its parameter bindings and local variables. When the call finishes, the caller receives the returned result, but the local workbench is no longer directly accessible.

Local scope helps because:

- internal names do not accidentally interfere with unrelated code;
- different functions can use the same convenient local name;
- a component can be developed and tested independently;
- the interface controls what information enters and leaves.

2.5.1 Local variables stay inside their function

```
# Define a function with a local variable named stress.
```

```
def calculate_stress(force, area):
    # This stress name exists locally during the call.
    stress = force / area

    # Return its value to the caller.
    return stress

# Store the returned value under a caller-side name.
result = calculate_stress(5000, 0.002)

# This line works because result exists in the caller's scope.
print(result)

# BUG: stress is local to the function and is not defined here.
print(stress)
```

Code 2.10: A local variable cannot be used directly outside its function.

The final line raises `NameError`. The correct communication channel is the returned value, here stored as `result`.

2.5.2 Caller names and parameter names may differ

```
# Caller-side variables include units in their names.
force_n = 5000
area_m2 = 0.002

# The function uses shorter local parameter names.
def calculate_stress(force, area):
    # The values are transferred; the caller's names are not copied.
    stress = force / area
    return stress

# Bind force_n's value to force and area_m2's value to area.
result = calculate_stress(force_n, area_m2)
```

Code 2.11: Binding caller values to differently named parameters.

Parameter binding transfers values into the call. The parameter name need not match the caller's variable name.

2.5.3 Two functions may safely use the same local name

```
# Calculate stress using a local result variable.
def calculate_stress(force, area):
    # This result exists only in this stress-function call.
    result = force / area
    return result

# Calculate current using another local result variable.
def calculate_current(voltage, resistance):
    # This result is independent of the result above.
    result = voltage / resistance
    return result
```

Code 2.12: Independent local variables with the same name.

There is no conflict. Each call receives its own local scope.

2.6 Prefer explicit inputs to hidden dependencies

A hidden dependency occurs when a function relies on information not visible in its parameter list.

```
# Store area outside the function.
area = 0.002  # m^2

# Define a function whose interface mentions only force.
def calculate_stress(force):
    # HIDDEN DEPENDENCY: area is required but not a parameter.
    return force / area
```

Code 2.13: A function with a hidden global dependency.

Problems include:

- the caller cannot see that area is required;
- changing the outside variable silently changes the function result;
- testing several areas requires changing global state;
- reuse in another file or notebook becomes fragile.

A better design is explicit:

```
# Expose both required engineering quantities in the interface.
def calculate_stress(force, area):
    # Return stress from the values supplied for this call.
    return force / area
```

Code 2.14: The required quantities are explicit parameters.

Design principle

A good engineering function should usually receive the quantities it needs through parameters. Explicit dependencies are easier to understand, control, document, test, and reuse.

2.7 Returning two related results

A function may return multiple related values when that produces a useful interface.

```
# Return two related engineering results for one loaded component.
def stress_and_fos(force, area, strength):
    # Calculate uniform axial stress in Pa.
    stress = force / area

    # Calculate the dimensionless strength-to-stress ratio.
    factor_of_safety = strength / stress

    # Return both values together.
```

```
    return stress, factor_of_safety

# Unpack the two returned values into separate caller-side names.
stress, fos = stress_and_fos(
    force=10000,      # N
    area=0.0005,      # m^2
    strength=250e6    # Pa
)

# Communicate both results.
print(f"Stress = {stress / 1e6:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
```

Code 2.15: Returning stress and factor of safety together.

Python packages the returned values together and the caller unpacks them. Tuples are studied formally in Week 5; the present emphasis is the interface: two results that belong to one calculation can be returned together.

Do not return many unrelated values merely because Python permits it. Multiple outputs should represent a coherent result.

2.8 Functions can contain engineering decisions

A function body may contain branching:

```
# Classify one temperature relative to stated lower and upper limits.
def classify_temperature(temp_c, low_c, high_c):
    # Temperatures below the lower limit are too low.
    if temp_c < low_c:
        return "too low"

    # The upper boundary is included in the normal interval.
    elif temp_c <= high_c:
        return "normal"

    # Every remaining value is above the upper limit.
    else:
        return "too high"
```

Code 2.16: A focused temperature-classification function.

The function has one clear job: classify a temperature. It does not also ask the user for input, calculate heat transfer, save a file, and print a long report.

Boundary behavior must be deliberate:

- at `temp_c == low_c`, the first condition is false and the result is `"normal"`;
- at `temp_c == high_c`, the `<=` condition is true and the result is `"normal"`.

2.9 Functions can contain loops without exposing the loop

```
# Return the number of digit characters in an equipment code.
```

```
def count_digits(code):
    # Begin with no digits counted.
    count = 0

    # Inspect each character in the string.
    for character in code:
        # Increase the counter only for digit characters.
        if character.isdigit():
            count += 1

    # Return the completed count after the loop.
    return count

# The caller needs the result, not the loop details.
number = count_digits("PUMP-107A")
print(number)
```

Code 2.17: Counting digit characters inside a function.

The function returns 3. The caller uses the abstraction `count_digits(code)`. The loop is an implementation detail inside the function.

2.10 A docstring records engineering meaning

A **docstring** is a string placed at the beginning of a function body to document the function.

```
# Define a documented axial-stress calculation.
def calculate_stress(force, area):
    """
    Return uniform axial stress in pascals.

    force: applied axial force in newtons
    area: cross-sectional area in square metres

    Assumption: the load is axial and the stress is
    uniformly distributed over the area.
    """
    # Apply sigma = F/A using the documented units.
    return force / area
```

Code 2.18: Engineering docstring for a stress function.

A useful engineering docstring states:

- purpose;
- input meaning and units;
- returned quantity and unit;
- important assumptions;
- valid ranges or failure behavior when relevant.

A docstring should not merely repeat the function name. It should reveal information that the code alone cannot express clearly, especially units and model assumptions.

2.11 A specification is a contract

A contract has two sides.

2.11.1 Caller assumptions

The caller must satisfy conditions such as:

- acceptable input types;
- required physical units;
- valid numerical ranges;
- required ordering of lower and upper limits;
- model assumptions.

2.11.2 Function guarantee

When the assumptions hold, the function promises:

- a defined returned quantity;
- a stated output type and unit;
- specified accuracy or tolerance, when relevant;
- a special failure indicator or raised error, if documented.

Engineering check

A function may satisfy its contract while the contract itself is unsuitable for the real engineering problem. A uniform-stress function can be perfectly correct for its simplified model but inappropriate for bending, buckling, fatigue, stress concentration, or a nonuniform cross-section.

2.12 Optional type hints can strengthen readability

Type hints document expected kinds of Python values:

```
# Type hints document expected Python value types.
def calculate_stress(
        force: float,
        area: float) -> float:
    """Return stress in Pa from force in N and area in m^2."""
    # Return a floating-point quotient.
    return force / area
```

Code 2.19: Optional type hints for a numerical function.

Type hints can help readers and editors, but they do not:

- encode physical units;

- prove that area is positive;
- prove the model is appropriate;
- automatically guarantee numerical correctness.

They are optional in ENG 101. Clear names, docstrings, and tests remain essential.

2.13 Assertions make expectations executable

An assertion checks a Boolean condition.

```
# Define a reusable Ohm's-law calculation.
def calculate_current(voltage, resistance):
    # Return current in A for voltage in V and resistance in ohm.
    return voltage / resistance

# A 12 V supply across 4 ohm should produce exactly 3 A here.
assert calculate_current(12, 4) == 3

# Zero voltage across a nonzero resistance should produce zero current.
assert calculate_current(0, 10) == 0
```

Code 2.20: Simple exact-value assertions for current.

If an assertion condition is true, execution continues silently. If false, Python raises `AssertionError`.

Assertions are useful for testing because clear inputs and returned outputs let one component be checked independently of a full program.

An assertion does not make an incorrect expected result correct. The expected value must come from an independent calculation, specification, or known property rather than from copying the function's own logic.

2.14 Test more than one convenient example

For `classify_temperature(temp, low, high)`, a useful test matrix is:

Test category	Example call values	Expected result	Why it matters
Representative low	5, 10, 30	too low	Normal use below the range
Lower boundary	10, 10, 30	normal	Tests inclusive lower boundary
Representative normal	20, 10, 30	normal	Interior of accepted interval
Upper boundary	30, 10, 30	normal	Tests inclusive upper boundary
Representative high	35, 10, 30	too high	Normal use above the range
Questionable limits	20, 30, 10	assumptions violated	Lower limit exceeds upper limit

The current function does not detect reversed limits. Therefore the contract must either require `low_c <= high_c`, or the implementation should be extended to validate this condition.

A test suite should distinguish:

Representative cases
Typical inputs from each intended category.

Boundary cases
Inputs exactly at or immediately around decision thresholds.

Invalid cases
Inputs that violate the stated contract.

Extreme cases
Very large, very small, zero, or negative values when physically relevant.

2.15 Floating-point functions need tolerance-based tests

Decimal floating-point values are often approximate. A tolerance-based comparison is usually safer than exact equality.

```
# Convert Celsius temperature to kelvin.
def celsius_to_kelvin(temp_c):
    # Add the exact scale offset used by this instructional calculation.
    return temp_c + 273.15

# Calculate the actual result for 25 deg C.
actual = celsius_to_kelvin(25.0)

# State the independently expected result.
expected = 298.15

# Choose an illustrative numerical tolerance for this test.
tolerance = 1e-9

# Pass when the absolute difference is sufficiently small.
assert abs(actual - expected) < tolerance
```

Code 2.21: Testing a floating-point conversion with a tolerance.

The tolerance should come from the purpose of the test, numerical method, measurement resolution, or required engineering accuracy. `1e-9` is illustrative, not universally appropriate.

Engineering check

Formatting a result to two decimal places does not change its stored value and is not a substitute for a justified tolerance. Display precision and acceptance tolerance answer different questions.

2.16 Common function failures and their clues

Symptom	Likely cause	First check
Nothing appears	Function defined but not called	Locate the call
`None` used in arithmetic	Printed instead of returned	Inspect the return path
Physically absurd number	Wrong argument order or units	Trace parameter binding
`NameError` outside function	Local variable used externally	Use the returned value
Result changes unexpectedly	Hidden global dependency	List all required inputs
Code after a point never runs	`return` executed too early	Trace control flow
Function difficult to test	Too many unrelated responsibilities	Split the job

2.16.1 Premature return example

```
# Attempt to count all positive values.
def count_positive(values):
    # Begin with no positive values counted.
    count = 0

    # Inspect each supplied value.
    for value in values:
        if value > 0:
            count += 1

        # BUG: returning inside the loop ends the function after one pass.
        return count
```

Code 2.22: A premature return inside a loop.

A correct return belongs after the loop:

```
# Count every positive value in the supplied sequence.
def count_positive(values):
    # Initialize the event counter.
    count = 0

    # Process every value before returning.
    for value in values:
```

```
        if value > 0:
            count += 1

    # Return only after the loop has finished.
    return count
```

Code 2.23: Correct return after all loop passes.

2.17 Debugging activity: four distinct problems

Study the deliberately faulty program:

```
# Store an area outside the function.
area_mm2 = 500

# Define a function that accepts force and strength.
def bar_check(force, strength):
    # BUG 1: hidden dependency on the outside area_mm2 variable.
    # BUG 2: mm^2 to m^2 requires division by 1,000,000, not 1000.
    area_m2 = area_mm2 / 1000

    # Calculate stress using the incorrectly converted area.
    stress = force / area_m2

    # BUG 3: strength is accepted but never used.
    # BUG 4: the function prints stress instead of returning it.
    print(stress)

# Because the function returns nothing, stress becomes None.
stress = bar_check(10000, 250e6)

# BUG CONSEQUENCE: dividing by None raises TypeError.
fos = 250e6 / stress
print(fos)
```

Code 2.24: A bar-check function containing four different design defects.

A focused repair separates the tasks and returns values explicitly:

```
# Convert an area from mm^2 to m^2.
def convert_mm2_to_m2(area_mm2):
    # 1 m^2 equals 1,000,000 mm^2.
    return area_mm2 / 1_000_000

# Return uniform stress in Pa.
def calculate_stress(force_n, area_m2):
    # Apply sigma = F/A.
    return force_n / area_m2

# Return factor of safety from strength and stress in matching units.
def calculate_fos(strength_pa, stress_pa):
    # The ratio is dimensionless.
    return strength_pa / stress_pa

# Execute the visible calculation chain.
area_m2 = convert_mm2_to_m2(500)
```

```
stress_pa = calculate_stress(10000, area_m2)
fos = calculate_fos(250e6, stress_pa)

# Report the returned values.
print(f"Stress = {stress_pa / 1e6:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
```

Code 2.25: One corrected and decomposed bar calculation.

2.18 Function interface checklist

Before implementation, ask:

- Can I state the job in one sentence?
- Is the name specific and meaningful?
- Are all required inputs parameters?
- Do parameter names reveal physical meaning?
- What exactly should be returned?

Before reuse, ask:

- Are units and assumptions documented?
- Have representative and boundary cases been tested?
- Can a teammate call the function without reading the body?
- Is the output suitable for later calculations?
- Is failure reported honestly?

2.19 Lecture 11 summary

Section summary

A trustworthy function has an explicit interface, local implementation details, documented assumptions, a reusable returned value, and independent evidence from tests. Positional arguments are compact; keyword arguments improve visibility; defaults express documented policies. Local scope protects components, while explicit parameters avoid hidden dependencies. Docstrings and contracts state meaning beyond syntax. Assertions and tolerance-based tests turn expectations into executable evidence.

2.20 Lecture 11 self-check

2.20.1 Design a density contract

For `calculate_density(mass_kg, volume_m3)`, provide two assumptions, a guarantee, one representative test, and one invalid input not handled by the simple function.

Possible assumptions are: `mass_kg` and `volume_m3` are numerical values expressed in kilograms and cubic metres, and `volume_m3 > 0`. The guarantee is that the function returns m/V in kg/m^3. A representative test is $2.7/0.001 = 2700$. An invalid input is zero volume, which would cause division by zero unless validated.

```
# Return density for physically meaningful positive volume.
def calculate_density(mass_kg, volume_m3):
    """
    Return density in kg/m^3.

    Assumptions: mass is in kg and volume is in m^3;
    volume must be positive.
    """
    # Apply rho = m/V.
    return mass_kg / volume_m3

# Verify a hand-checkable representative case.
assert calculate_density(2.7, 0.001) == 2700
```

Code 2.26: A documented density function with a representative test.

2.20.2 Boundary reasoning

For `classify_temperature(temp_c, low_c, high_c)`, what is returned at exactly `low_c` and exactly `high_c`?

Both boundaries are returned as `"normal"`. At the lower boundary, `temp_c < low_c` is false and the `elif` condition is true. At the upper boundary, `temp_c <= high_c` is true. This behavior follows from the inclusive `<=` comparison and must match the intended specification.

3 Lecture 12: Decomposition and Abstraction in Engineering Programs

Lecture-slide alignment: Lecture 12 of the Week 4 slides: identifying function boundaries; decomposition into smaller questions; one clear responsibility; decomposition tables and data flow; top-down planning and bottom-up confidence; readable workflows; abstraction and stable interfaces; naming; separation of layers; refactoring bisection; precise numerical contracts; bar-design functions and orchestration; separate reporting; component and integration tests; small libraries; Assignment 2 preparation; scope boundary; concept map; and closing activities.

Learning outcomes

By the end of this chapter, you should be able to:

1. explain decomposition as dividing a larger problem into smaller tasks;
2. identify function candidates from an engineering procedure;
3. design functions with one clear responsibility;
4. create a decomposition table before writing code;
5. represent data flow among functions;
6. combine top-down planning with bottom-up testing;
7. explain abstraction through a clear and stable interface;
8. separate input, calculation, decision, and reporting layers;
9. refactor a Week 3 numerical search into a reusable function;
10. state a precise contract for an approximate numerical result;
11. build a function-based bar-design report;
12. test components before integrating the full program;
13. outline a small engineering calculation library;
14. prepare for Assignment 2 using Moodle's authoritative instructions.

3.1 Retrieval warm-up: draw the function boundaries

Consider the following correct but monolithic script:

```
# Store the engineering inputs.
force_n = 10000          # N
area_mm2 = 500           # mm^2
strength_mpa = 250       # MPa
required_fos = 2.0       # dimensionless

# Convert the area from mm^2 to m^2.
area_m2 = area_mm2 / 1e6

# Calculate uniform axial stress.
```

```
stress_pa = force_n / area_m2
stress_mpa = stress_pa / 1e6

# Convert strength to Pa and calculate factor of safety.
strength_pa = strength_mpa * 1e6
fos = strength_pa / stress_pa

# Classify the design using the stated required factor of safety.
status = "acceptable" if fos >= required_fos else "not acceptable"

# Format and display the report.
print(f"Stress = {stress_mpa:.2f} MPa")
print(f"Factor of safety = {fos:.2f}")
print(f"Status = {status}")
```

Code 3.1: A complete calculation written as one long sequence.

Possible function candidates are:

1. convert area units;
2. calculate stress;
3. calculate factor of safety;
4. classify the design;
5. format the report.

The program may be short, but these operations have different responsibilities and different testing needs.

3.2 Decomposition: turn one large task into smaller questions

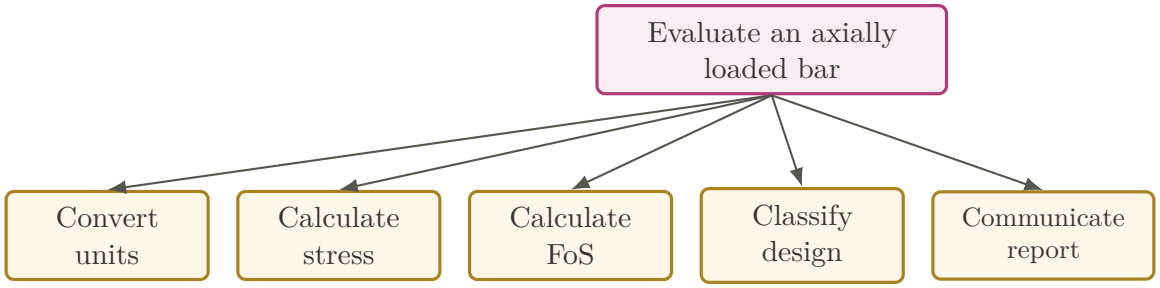
Definition

Decomposition is the process of dividing a larger problem into smaller, more focused tasks that can be understood, implemented, tested, and changed with limited dependence on one another.

The high-level problem

Evaluate an axially loaded bar

can be decomposed as:



Each subtask should have a clear question:

- What area in square metres corresponds to the supplied square-millimetre value?
- What stress follows from the force and area?
- What factor of safety follows from material strength and stress?
- Does that factor meet the stated requirement?
- How should the results be communicated to a human reader?

3.3 The guiding question: what is the job of this function?

A focused function can be described with one clear verb and object:

```
# Calculate one engineering quantity from two explicit inputs.
def calculate_stress(force, area):
    # Apply one model and return one result.
    return force / area
```

Code 3.2: A focused calculation function.

By contrast, an overloaded function might:

```
# BAD DESIGN: the name and empty interface reveal almost nothing.
def do_everything():
    # This placeholder represents input, conversion, calculations,
    # classifications, plots, file output, and printing in one place.
    pass
```

Code 3.3: A deliberately vague and overloaded function design.

An overloaded function is hard to reuse, test, or diagnose because it performs unrelated work.

Design principle

If the function's job description requires several unrelated verbs joined by "and," consider splitting the function. Coordination is a legitimate job, but the coordinated calculation components should still remain focused.

3.4 A decomposition table comes before code

A decomposition table lets us review names, responsibilities, and outputs before any function body is written.

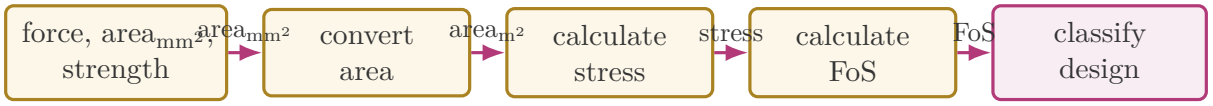
Function	Single responsibility	Returned value
`convert_mm2_to_m2`	Convert area units	area in m^2
`calculate_stress`	Apply $\sigma = F/A$	stress in Pa
`calculate_fos`	Compute strength divided by stress	dimensionless factor of safety
`classify_design`	Compare FoS with a requirement	status string
`format_design_report`	Create human-readable text	report string

Design before syntax helps reveal:

- missing inputs;
- unclear units;
- responsibilities that overlap;
- output types that do not match the next task;
- vague names.

3.5 Data flow connects the components

The output of one function often becomes the input of another:



Interfaces must meet. The output of one function should have the meaning, type, and unit expected by the next function.

A program may fail even when each local formula is correct if one function returns megapascals while the next assumes pascals. Data flow therefore includes unit flow.

3.6 Top-down design and bottom-up confidence

3.6.1 Top-down: plan the system

Top-down design begins with the engineering question:

1. state the problem and required outputs;
2. list the major steps;
3. assign one responsibility to each function;
4. define each interface and the data flow among functions.

3.6.2 Bottom-up: build trust

Bottom-up development builds and tests the pieces:

1. implement one small function;
2. test it with a known case;
3. connect two trusted functions;
4. test the integrated data flow;
5. add the next component;
6. test the complete workflow and interpret the result.

Do not write five untested functions, connect them all, and only then begin debugging. Component tests localize defects; integration tests show whether meanings, types, units, and order match between components.

3.7 A well-decomposed program reads like an engineering procedure

The main workflow can be compact and meaningful:

```
# Convert the input area to the unit required by the stress model.
area_m2 = convert_mm2_to_m2(area_mm2)

# Calculate stress from the applied force and converted area.
stress_pa = calculate_stress(force_n, area_m2)

# Calculate factor of safety from material strength and stress.
fos = calculate_factor_of_safety(strength_pa, stress_pa)

# Apply the stated acceptance policy.
status = classify_design(fos, required_fos)

# Create a human-readable report string.
report = format_design_report(stress_pa, fos, status)

# Display the report after the calculations are complete.
print(report)
```

Code 3.4: Calling focused functions in engineering order.

Read aloud, the code mirrors the engineering reasoning:

> Convert the area, calculate stress, calculate factor of safety, classify the design, format the report, and print it.

This is a major benefit of good names and decomposition.

3.8 Abstraction: use the interface, hide repeated detail

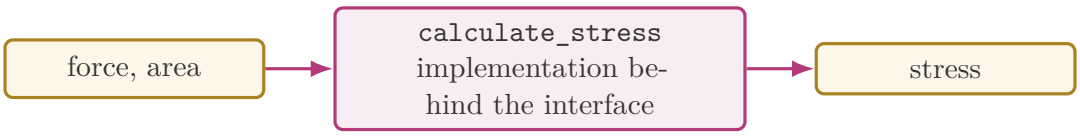
Definition

Abstraction preserves the information needed to use a component while hiding internal details that the caller should not need to repeat on every use.

For `calculate_stress(force, area)`, the caller needs:

- the purpose of the function;
- the required arguments and units;
- the returned quantity and unit;
- the assumptions and failure behavior.

The caller does not need to rewrite `force / area` at every use. In more complex functions, the hidden implementation may contain many statements, loops, conditionals, or library calls.



3.9 Implementation can change while the interface remains stable

Imagine three versions of `calculate_stress(force, area)`:

Version A
A direct scalar formula.

Version B
Additional validation and clearer failure reporting.

Version C
A NumPy implementation that accepts arrays of forces or areas.

Caller code may remain:

```
# Call the documented interface without depending on its internal version.
stress = calculate_stress(force, area)
```

Code 3.5: A stable call despite possible internal changes.

This stability is valuable only if each implementation satisfies the same documented contract. An internal change that silently alters units or meaning is not a valid implementation replacement.

3.10 Good abstraction depends on good names

Weak names hide purpose:

```python
# BAD DESIGN: the purpose and units are hidden.
def f(x, y):
    # Return a quotient with no stated engineering meaning.
    return x / y

# The caller must reconstruct what a, b, and z represent.
z = f(a, b)
```

Code 3.6: Weak abstraction with uninformative names.

Strong names communicate the operation:

```python
# Return uniform axial stress from force and area.
def calculate_stress(force, area):
    # Apply the documented stress model.
    return force / area

# The workflow is understandable without opening the body.
stress = calculate_stress(force, area)
```

Code 3.7: Strong abstraction with meaningful names.

A practical naming test is:

> Can a teammate understand the main workflow without opening the function body?

3.11 Separate computation from input and reporting

A useful layered design is:

Layer	Responsibility	Example
Input layer	Read and convert user input	`float(input(...))`
Calculation layer	Apply formulas and numerical algorithms	`calculate_stress(...)`
Decision layer	Apply stated engineering rules	`classify_design(...)`
Presentation layer	Format and display results	`format_report(...)`

Why separate the layers?

- the calculation can be tested without typing interactive input;
- the same function can be used in a notebook, script, graphical interface, or automated test;
- numerical results can be printed, plotted, written to a file, or passed onward without recomputing them;
- changes in presentation do not disturb the engineering model.

```python
# Define a reusable calculation with no input() or print() inside.
def calculate_energy_kwh(power_kw, time_h):
    # Return energy in kWh.
    return power_kw * time_h

```

```
# Input layer: receive and convert user text.
power_kw = float(input("Power in kW: "))
time_h = float(input("Operating time in h: "))

# Calculation layer: obtain the reusable numerical result.
energy_kwh = calculate_energy_kwh(power_kw, time_h)

# Presentation layer: communicate the result.
print(f"Energy = {energy_kwh:.2f} kWh")
```

Code 3.8: Calculation separated from interactive input and presentation.

3.12 Refactor Week 3: package bisection as a reusable function

Week 3 developed bisection as a script. Week 4 packages the algorithm behind an interface.

```
# Approximate the nonnegative square root of a nonnegative value.
def square_root_bisection(value, tolerance=1e-6):
    """
    Return a nonnegative square-root approximation.

    Assumptions:
    - value is a nonnegative int or float;
    - tolerance is positive;
    - floating-point arithmetic is adequate.
    """
    # Start with a bracket containing the nonnegative root.
    low = 0.0
    high = max(1.0, value)

    # Evaluate the first midpoint.
    midpoint = (low + high) / 2

    # Continue while the squared-value residual is too large.
    while abs(midpoint**2 - value) >= tolerance:
        # If the midpoint square is too small, keep the right half.
        if midpoint**2 < value:
            low = midpoint
        else:
            # Otherwise keep the left half.
            high = midpoint

        # Recompute the midpoint of the retained bracket.
        midpoint = (low + high) / 2

    # Return the accepted approximation.
    return midpoint
```

Code 3.9: Reusable bisection approximation for a nonnegative square root.

The bracket variables, loop, midpoint updates, and stopping test move inside the function. The caller sees only the target and optional tolerance.

3.12.1 Reuse the numerical method

```python
# Use the default residual tolerance for sqrt(2).
root_2 = square_root_bisection(2.0)

# Use the default residual tolerance for sqrt(10).
root_10 = square_root_bisection(10.0)

# Request a tighter residual tolerance for sqrt(25).
root_25 = square_root_bisection(
    25.0,
    tolerance=1e-10
)

# Display the three returned approximations.
print(root_2)
print(root_10)
print(root_25)
```

Code 3.10: Several calls to the same bisection abstraction.

The interval-halving details are written and tested once. The caller states the value and accuracy request.

Engineering check

The displayed implementation has a documented assumption that `value` is nonnegative. It also lacks a maximum-iteration guard and explicit validation of a positive tolerance. Later work on exceptions and systematic testing will strengthen failure handling.

3.13 Specify the numerical abstraction precisely

For `square_root_bisection(value, tolerance)`, the slides state assumptions such as:

- `value` is an `int` or `float`;
- `value >= 0`;
- `tolerance > 0`;
- floating-point arithmetic is adequate for the task.

The guarantee is:

$$\text{return a nonnegative } r \text{ such that} \qquad |r^2 - \text{value}| < \text{tolerance}.$$

Do not promise more. This contract controls the residual in the squared equation. It does not directly guarantee an exact error bound on r for every input, nor does it validate every possible data type or numerical condition.

Function contract

A precise numerical contract states which error quantity is controlled. “Returns an accurate answer” is vague. “Returns $r \geq 0$ satisfying $|r^2 - x| < \varepsilon$ for nonnegative x and positive ε” is testable.

3.14 Bar-design functions: calculations and decision

We now implement the decomposition table.

```
# Convert area from square millimetres to square metres.
def convert_mm2_to_m2(area_mm2):
    # 1 m^2 = 1,000,000 mm^2.
    return area_mm2 / 1e6

# Calculate uniform axial stress in pascals.
def calculate_stress(force_n, area_m2):
    # Apply sigma = F/A.
    return force_n / area_m2

# Calculate dimensionless factor of safety.
def calculate_fos(strength_pa, stress_pa):
    # Use matching pressure units in numerator and denominator.
    return strength_pa / stress_pa

# Classify the factor of safety using a stated requirement.
def classify_design(fos, required_fos=2.0):
    # Include equality in the acceptable category.
    if fos >= required_fos:
        return "ACCEPTABLE"

    # Handle every lower value.
    return "NOT ACCEPTABLE"
```

Code 3.11: Focused bar-design calculation and decision functions.

These responsibilities can be tested separately:

- unit conversion;
- formula evaluation;
- ratio calculation;
- policy-based classification.

3.15 The integrated calculation function

An **orchestration function** coordinates focused components without replacing them.

```
# Coordinate the complete numerical bar evaluation.
def evaluate_bar(
        force_n,
        area_mm2,
        strength_pa,
        required_fos=2.0):
    # Convert the supplied area to the stress model's unit.
    area_m2 = convert_mm2_to_m2(area_mm2)

    # Calculate stress from the converted area.
    stress_pa = calculate_stress(force_n, area_m2)

```

```
    # Calculate the factor of safety.
    fos = calculate_fos(strength_pa, stress_pa)

    # Apply the stated acceptance rule.
    status = classify_design(fos, required_fos)

    # Return all engineering results needed by the caller.
    return stress_pa, fos, status

# Evaluate one bar using keyword arguments.
stress_pa, fos, status = evaluate_bar(
    force_n=10000,       # N
    area_mm2=500,        # mm^2
    strength_pa=250e6    # Pa
)
```

Code 3.12: An orchestration function for the bar workflow.

The function's job is coordination. It returns the results and does not decide how they must be displayed.

3.16 Keep reporting separate from calculation

```
# Build a human-readable bar-design report string.
def format_bar_report(stress_pa, fos, status):
    # Convert stress to MPa for presentation.
    stress_mpa = stress_pa / 1e6

    # Return one multi-line string rather than printing inside the function.
    return (
        "BAR DESIGN REPORT\n"
        f"Stress: {stress_mpa:.2f} MPa\n"
        f"Factor of safety: {fos:.2f}\n"
        f"Status: {status}"
    )

# Format the already calculated results.
report = format_bar_report(stress_pa, fos, status)

# Display the report in the presentation layer.
print(report)
```

Code 3.13: Formatting a report without rerunning the calculations.

The same numerical results could instead be:

- written to a file;
- plotted;
- stored in a collection;
- passed to another decision function;
- used in a design comparison.

No formula needs to be rerun merely to change the presentation.

Engineering check

The word "ACCEPTABLE" refers only to the stated uniform-stress model and required factor of safety. It does not certify fatigue, buckling, yielding under multiaxial stress, stress concentration, manufacturing defects, or any other failure mode not represented by the program.

3.17 Test pieces before testing the whole program

Component tests build a hierarchy of evidence.

```
# Test the unit conversion with an exact known relationship.
assert convert_mm2_to_m2(1_000_000) == 1

# Test the stress formula with 10 N / 2 m^2 = 5 Pa.
assert calculate_stress(10, 2) == 5

# Test the factor-of-safety ratio with 100 / 20 = 5.
assert calculate_fos(100, 20) == 5

# Test the inclusive classification boundary.
assert classify_design(2.0, 2.0) == "ACCEPTABLE"

# Test the integrated data flow using hand-checkable values.
stress, fos, status = evaluate_bar(
    force_n=10,             # N
    area_mm2=2_000_000,     # equals 2 m^2
    strength_pa=20,         # Pa
    required_fos=2.0
)

# Verify the integrated results.
assert stress == 5
assert fos == 4
assert status == "ACCEPTABLE"
```

Code 3.14: Component tests followed by an integration test.

A useful hierarchy is:

1. unit conversion;
2. each physical formula;
3. boundary decision;
4. integrated data flow;
5. physical interpretation and model suitability.

Component tests localize defects. If every component test passes but the integrated result fails, investigate the order, units, or values passed between functions.

3.18 A small engineering library collects reusable functions

At first, several functions may live in one notebook or Python file:

```python
# engineering_calculations.py
# Store focused, documented, tested functions in one file.

def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa."""
    return force_n / area_m2


def calculate_current(voltage_v, resistance_ohm):
    """Return current in A from voltage and resistance."""
    return voltage_v / resistance_ohm


def calculate_power(voltage_v, current_a):
    """Return electrical power in W."""
    return voltage_v * current_a
```

Code 3.15: Outline of a small engineering calculation file.

Later, a program may import selected tools:

```python
# Import only the functions needed by the current program.
from engineering_calculations import (
    calculate_stress,
    calculate_power
)
```

Code 3.16: Preview of importing selected functions from a module.

Modules are a later organizational step. The foundation is a set of focused, documented, tested functions.

3.19 Activity: decompose an energy-cost calculator

A device has power in watts, operating time in hours, and electricity price in KZT per kilowatt-hour. The program must report energy in kilowatt-hours and cost.

A possible decomposition is:

- `watts_to_kilowatts`;
- `calculate_energy_kwh`;
- `calculate_cost`;
- `format_energy_report`.

One complete solution is:

```python
# Convert power from W to kW.
def watts_to_kilowatts(power_w):
    # Divide by 1000 because 1 kW = 1000 W.
```

```
    return power_w / 1000

# Calculate energy in kWh.
def calculate_energy_kwh(power_kw, time_h):
    # Apply E = P*t using kW and h.
    return power_kw * time_h

# Calculate cost in KZT.
def calculate_cost(energy_kwh, price_kzt_per_kwh):
    # Multiply energy by the unit price.
    return energy_kwh * price_kzt_per_kwh

# Format a reusable report string.
def format_energy_report(energy_kwh, cost_kzt):
    # Return clear labels, units, and controlled decimal places.
    return (
        "ENERGY COST REPORT\n"
        f"Energy: {energy_kwh:.2f} kWh\n"
        f"Cost: {cost_kzt:.2f} KZT"
    )

# Store one operating case.
power_w = 1500.0              # W
time_h = 2.0                  # h
price_kzt_per_kwh = 30.0      # KZT/kWh

# Execute the calculation workflow.
power_kw = watts_to_kilowatts(power_w)
energy_kwh = calculate_energy_kwh(power_kw, time_h)
cost_kzt = calculate_cost(energy_kwh, price_kzt_per_kwh)
report = format_energy_report(energy_kwh, cost_kzt)

# Display the completed report.
print(report)
```

Code 3.17: Decomposed energy-use and cost program.

Known tests include:

```
# Verify the W-to-kW conversion.
assert watts_to_kilowatts(1000) == 1

# Verify that 2 kW for 3 h produces 6 kWh.
assert calculate_energy_kwh(2, 3) == 6

# Verify that 6 kWh at 30 KZT/kWh costs 180 KZT.
assert calculate_cost(6, 30) == 180
```

Code 3.18: Known tests for the energy-cost components.

3.20 Pair design review: can another engineer use the function?

The builder proposes interfaces in a chosen context such as mechanics, circuits, fluids, energy, or sensors. Before writing bodies, the builder should:

- propose four function interfaces;
- include units in names or documentation;
- state at least one assumption;
- identify each returned value;
- write one docstring.

The reviewer, without seeing the implementation, asks:

- Can I call it correctly?
- Is any dependency hidden?
- Is the output unambiguous?
- What test would I run first?

This review tests the quality of the interface rather than the complexity of the body.

3.21 Assignment 2: function-based problem solving

Use the Assignment 2 document on Moodle for the exact task, deadline, permitted resources, submission format, marking criteria, and AI-use rules. The slides establish the preparation themes but do not replace the authoritative assignment brief.

Course-aligned preparation includes being able to:

- identify appropriate functions from a problem statement;
- define clear parameters and returned values;
- use meaningful names;
- document units and assumptions;
- test small components;
- combine functions into an engineering workflow;
- explain, reproduce, test, and modify every submitted function.

3.22 Week 4 scope boundary

Developed this week:

- function definitions and calls;
- parameters, arguments, and returns;
- local scope;

- documentation and simple tests;
- decomposition and abstraction.

Developed later:

- tuples and lists in Week 5;
- recursion and dictionaries in Week 6;
- systematic testing, exceptions, and assertions in Week 7;
- classes and objects in Weeks 8–9.

Strong function design is the foundation for these later topics.

3.23 Lecture 12 summary

Section summary

Decomposition divides a larger problem into focused tasks. A decomposition table clarifies names, responsibilities, inputs, outputs, and units before coding. Data flow connects the functions, so interfaces must agree. Top-down design plans the system; bottom-up testing builds confidence. Abstraction lets callers use documented interfaces without repeating internal details. Calculation, decision, input, and reporting should be separated when possible. An orchestration function may coordinate focused components, and component tests should precede integration tests.

3.24 Lecture 12 self-check

3.24.1 Decompose a projectile calculation

A projectile model must calculate height and vertical velocity at a specified time and print a report. Using

$$h(t) = h_0 + v_0 t - \frac{1}{2} g t^2, \qquad v(t) = v_0 - g t,$$

propose two calculation functions, one reporting function, the data passed among them, and one test for each calculation.

```python
# Calculate height in m at time t in s.
def calculate_height(h0_m, v0_mps, time_s, g_mps2=9.81):
    # Apply constant-acceleration vertical motion.
    return h0_m + v0_mps * time_s - 0.5 * g_mps2 * time_s**2

# Calculate vertical velocity in m/s at time t in s.
def calculate_velocity(v0_mps, time_s, g_mps2=9.81):
    # Apply v = v0 - g*t.
    return v0_mps - g_mps2 * time_s
```

```python

# Format height and velocity as a human-readable report.
def format_projectile_report(time_s, height_m, velocity_mps):
    # Return text rather than printing inside the calculation functions.
    return (
        f"Time: {time_s:.2f} s\n"
        f"Height: {height_m:.2f} m\n"
        f"Vertical velocity: {velocity_mps:.2f} m/s"
    )

# At t=0, height and velocity should equal their initial values.
assert calculate_height(30, 18, 0) == 30
assert calculate_velocity(18, 0) == 18

# Evaluate one later time and format the report.
time_s = 2.0
height_m = calculate_height(30, 18, time_s)
velocity_mps = calculate_velocity(18, time_s)
print(format_projectile_report(time_s, height_m, velocity_mps))
```

Code 3.19: One decomposed projectile solution.

3.24.2 Explain abstraction

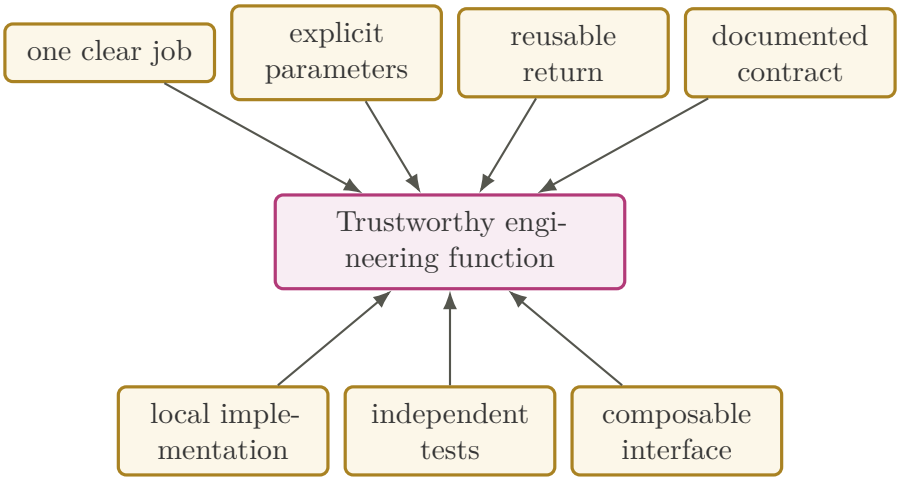
Why can the implementation of `calculate_stress(force, area)` change without requiring every caller to change?

Callers depend on the documented interface and contract rather than on internal statements. If the function keeps the same parameter meaning, returned quantity, units, and guarantees, its body may add validation, improve numerical implementation, or use a library while the call remains unchanged. If the meaning or contract changes, the interface change must be communicated.

4 Week 4 Synthesis and Practice

4.1 The Week 4 concept map

The three lectures form one coherent design story:



A function is trustworthy when its job is focused, its required information is explicit, its result is reusable, its assumptions and guarantee are documented, its internal names are protected by local scope, and its behavior is supported by tests. Decomposition creates these components; abstraction lets a larger program use them without repeating every internal detail.

4.2 Week 4 quick reference

4.2.1 Definitions, calls, and return values

Python pattern	Meaning
`def f(x):`	Define a function named `f` with parameter `x`
`return value`	Send a result to the caller and end the current call
`y = f(3)`	Call the function and store its returned result
`f(x=3)`	Supply an argument by parameter name
`def f(x, tol=0.01):`	Give `tol` a default value
`return a, b`	Return two related values together
`a, b = f(x)`	Unpack two returned values
`assert condition`	Continue if true; raise `AssertionError` if false

4.2.2 Engineering design habits

Habit	Meaning
One clear job	Keep each function focused on one coherent responsibility
Explicit inputs	Pass required quantities as parameters
Meaningful name	Reveal the engineering operation
Visible units	Use parameter names, docstrings, comments, and reports
Reusable return	Return values needed by later calculations
Local implementation	Keep temporary details inside the function
Documented contract	State assumptions and guarantees
Simple test	Begin with a case checked independently by hand
Boundary tests	Check exact thresholds and nearby values
Small pieces	Compose functions into a readable workflow
Separate layers	Keep input, calculation, decision, and presentation distinct

Key idea

For every function, ask four questions: What does it need? What does it return? What assumptions must hold? How will we know it works?

4.3 Glossary

Abstraction
Use of a component through its relevant interface while hiding repeated implementation details.

Argument
A value or expression supplied to a function during a call.

Assertion
An executable claim that raises `AssertionError` when its Boolean condition is false.

Bottom-up development
Implementing and testing small components before integrating them into a larger workflow.

Call
Use of a function by writing its name followed by parentheses and arguments.

Call frame
A conceptual temporary workspace containing parameter bindings and local variables for one function call.

Caller
The statement or program component that invokes a function.

Composition
Connecting functions so that one returned value becomes an input to another function or calculation.

Contract
The caller assumptions and function guarantee that define correct use.

Default argument
A parameter value used when the caller does not supply a replacement.

Definition
The `def` statement that records a function's name, parameters, and body.

Decomposition
Division of a larger problem into smaller, focused tasks.

Dependency
Information a function needs in order to produce its result.

Docstring
A string at the beginning of a function body that documents purpose, inputs, outputs, units, assumptions, or behavior.

Explicit dependency
Required information visible as a parameter or otherwise clearly stated in the interface.

Function
A named, reusable unit of computation.

Guarantee
What a function promises when the caller satisfies the contract assumptions.

Hidden dependency
Required information taken from outside the visible interface, often through global state.

Integration test
A test of connected components and their data flow.

Interface
The visible way to use a function: name, parameters, returned value, and documented meaning.

Keyword argument
An argument supplied using the parameter name, such as `area=0.002`.

Local scope
The region inside one function call where its parameters and local variables are available.

Orchestration function
A function whose coherent responsibility is to coordinate several focused components.

Parameter
A local name in a function definition that receives an argument value during a call.

Positional argument
An argument matched to a parameter according to order.

Premature return
A return executed before the intended work is complete, often because it is placed incorrectly inside a loop or branch.

Refactoring
Restructuring code to improve organization or reuse while preserving intended behavior.

Return value
The value sent from a function to its caller.

Scope
The region where a name can be used.

Single responsibility
A focused function job that can be explained as one coherent task.

Top-down design
Planning a system from the overall question through major steps, functions, interfaces, and data flow.

Type hint
Optional annotation that documents an expected Python type without encoding units or proving correctness.

Unit test
A focused test of one small function or component.

Unpacking
Assigning several returned values to several names in one statement.

4.4 Reading function code systematically

When you encounter an unfamiliar function, do not begin by reading every statement as an isolated line. Use the following order:

1. Read the function name and propose its job.
2. Read the parameter list and identify required information.
3. Read the docstring or nearby comments for units and assumptions.
4. Locate every `return` path.
5. Trace one small call by binding arguments to parameters.
6. Identify local variables and how they change.
7. Check decisions, loop stopping conditions, and early returns.
8. Compare the returned value with an independent expected result.

4.5 Cumulative practice: definitions, calls, and return values

4.5.1 Practice 1: trace a function call

Predict the output and explain each parameter binding.

```python
# Convert pressure from MPa to Pa.
def mpa_to_pa(pressure_mpa):
    # One MPa equals 1,000,000 Pa.
    pressure_pa = pressure_mpa * 1_000_000
    return pressure_pa

# Call the function for 2.5 MPa.
result = mpa_to_pa(2.5)

# Display the returned value.
print(result)
```

Code 4.1: Trace this pressure conversion without running it first.

The argument 2.5 is bound to the parameter `pressure_mpa`. The local variable `pressure_pa` becomes 2,500,000. The function returns that value, which is assigned to `result`. The printed output is `2500000.0`.

4.5.2 Practice 2: definition without a call

Why does this program display nothing?

```
# Define a valid temperature conversion.
def kelvin_to_celsius(temp_k):
    # Return temperature in deg C.
    return temp_k - 273.15

# No call appears below this definition, so no result is calculated or printed.
```

Code 4.2: A function definition that is never called.

The definition records the tool but does not use it. Add a call and a presentation statement:

```
# Define the reusable conversion.
def kelvin_to_celsius(temp_k):
    # Convert K to deg C.
    return temp_k - 273.15

# Call the function for the freezing point of water.
temp_c = kelvin_to_celsius(273.15)

# Display the returned result.
print(f"Temperature = {temp_c:.2f} deg C")
```

Code 4.3: Calling and displaying the conversion.

4.5.3 Practice 3: print versus return

Repair the following function so that its result can be multiplied by time.

```
# Define a power calculation.
def power_bad(voltage_v, current_a):
    # Calculate power in W.
    power_w = voltage_v * current_a

    # BUG: this communicates to a human but returns None.
    print(power_w)

# energy_j cannot be calculated from None.
energy_j = power_bad(12, 3) * 10
```

Code 4.4: Faulty function that prints rather than returns.

```python
# Return electrical power in W.
def calculate_power(voltage_v, current_a):
    # Apply P = V*I.
    return voltage_v * current_a

# Reuse the returned power for a 10 s energy calculation.
energy_j = calculate_power(12, 3) * 10

# Display the final energy.
print(f"Energy = {energy_j:.1f} J")
```

Code 4.5: Correct power function with reusable return.

4.6 Cumulative practice: parameters, scope, and contracts

4.6.1 Practice 4: repair a hidden dependency

```python
# Store area outside the function.
area_m2 = 0.01

# Calculate volumetric flow rate from velocity.
def calculate_flow_rate(velocity_mps):
    # HIDDEN DEPENDENCY: area_m2 is required but not visible in the interface.
    return velocity_mps * area_m2
```

Code 4.6: A flow-rate function with a hidden area value.

Pass area explicitly:

```python
# Return volumetric flow rate in m^3/s.
def calculate_flow_rate(velocity_mps, area_m2):
    """
    Return Q = v*A in m^3/s.

    Inputs must be in m/s and m^2.
    """
    # Multiply mean velocity by cross-sectional area.
    return velocity_mps * area_m2

# Evaluate one pipe section explicitly.
flow_m3_s = calculate_flow_rate(2.0, 0.01)
print(f"Flow rate = {flow_m3_s:.3f} m^3/s")
```

Code 4.7: Explicit flow-rate interface.

4.6.2 Practice 5: write a contract

Write assumptions and a guarantee for:

```python
# Calculate travel time from distance and speed.
def calculate_time(distance_m, speed_mps):
    # Return time in seconds.
    return distance_m / speed_mps
```

Code 4.8: Function requiring a contract.

Possible assumptions: `distance_m` and `speed_mps` are numerical; distance is nonnegative; speed is positive; the speed is constant; and units are metres and metres per second. Guarantee: when assumptions hold, return `distance_m / speed_mps` in seconds. An important invalid input is zero speed.

4.6.3 Practice 6: local names

Predict whether the final line works.

```python
# Define a local conversion variable.
def convert_length(length_mm):
    # This name exists only during the function call.
    length_m = length_mm / 1000
    return length_m

# Store the returned value outside the function.
converted = convert_length(500)

# BUG: length_m is local and is not defined in caller scope.
print(length_m)
```

Code 4.9: A local variable outside its scope.

The final line raises `NameError`. The caller should print `converted`. The returned value crosses the interface; the local variable name does not.

4.7 Cumulative practice: testing

4.7.1 Practice 7: representative and boundary tests

Write tests for a function that returns `"low"` below 100 kPa, `"normal"` from 100 kPa through 200 kPa inclusive, and `"high"` above 200 kPa.

```python
# Classify pressure relative to inclusive normal boundaries.
def classify_pressure(pressure_kpa):
    # Values below 100 kPa are low.
    if pressure_kpa < 100:
        return "low"
```

```
6
7       # Values from 100 through 200 kPa are normal.
8       if pressure_kpa <= 200:
9           return "normal"
10
11      # Every remaining value is high.
12      return "high"
13
14  # Representative low case.
15  assert classify_pressure(80) == "low"
16
17  # Exact lower boundary.
18  assert classify_pressure(100) == "normal"
19
20  # Representative interior case.
21  assert classify_pressure(150) == "normal"
22
23  # Exact upper boundary.
24  assert classify_pressure(200) == "normal"
25
26  # Representative high case.
27  assert classify_pressure(250) == "high"
```

Code 4.10: Pressure classification with representative and boundary tests.

4.7.2 Practice 8: tolerance-based test

Write a tolerance-based test for a function that converts 1 bar to 100,000 Pa.

```
1  # Convert bar to pascals.
2  def bar_to_pa(pressure_bar):
3      # Use 1 bar = 100,000 Pa.
4      return pressure_bar * 100_000
5
6  # Evaluate the function and state the expected result.
7  actual = bar_to_pa(1.0)
8  expected = 100_000.0
9
10 # Use a small absolute tolerance for this deterministic conversion test.
11 tolerance_pa = 1e-9
12 assert abs(actual - expected) < tolerance_pa
```

Code 4.11: Tolerance-based pressure conversion test.

4.7.3 Practice 9: find the premature return

```
1  # Attempt to find the largest value.
2  def find_maximum(values):
3      # Begin with the first value as the current maximum.
4      maximum = values[0]
```

```
5
6       # Compare each value with the current maximum.
7       for value in values:
8           if value > maximum:
9               maximum = value
10
11          # BUG: returning here ends after the first pass.
12          return maximum
```

Code 4.12: A maximum search with a misplaced return.

Move the return outside the loop:

```
1  # Return the largest value in a nonempty sequence.
2  def find_maximum(values):
3      # Use the first value as the initial maximum.
4      maximum = values[0]
5
6      # Inspect every value before returning.
7      for value in values:
8          if value > maximum:
9              maximum = value
10
11     # Return after all comparisons are complete.
12     return maximum
13
14 # Verify a simple case.
15 assert find_maximum([3, 8, 5]) == 8
```

Code 4.13: Correct maximum search.

4.8 Cumulative practice: decomposition and abstraction

4.8.1 Practice 10: decompose a resistor report

Design a program that accepts voltage and resistance, calculates current and power, classifies whether the power exceeds a stated resistor rating, and formats a report.

One decomposition is:

- `calculate_current(voltage_v, resistance_ohm);`
- `calculate_power(voltage_v, current_a);`
- `classify_power(power_w, rating_w);`
- `format_resistor_report(...).`

```
1  # Return current in A from voltage in V and resistance in ohm.
2  def calculate_current(voltage_v, resistance_ohm):
3      # Apply Ohm's law.
```

```
    return voltage_v / resistance_ohm

# Return electrical power in W.
def calculate_power(voltage_v, current_a):
    # Apply P = V*I.
    return voltage_v * current_a

# Compare calculated power with a stated component rating.
def classify_power(power_w, rating_w):
    # Include equality as within the stated rating.
    if power_w <= rating_w:
        return "WITHIN STATED RATING"
    return "EXCEEDS STATED RATING"

# Build a human-readable report.
def format_resistor_report(voltage_v, resistance_ohm,
                           current_a, power_w, status):
    # Return text so the caller chooses where to display or store it.
    return (
        "RESISTOR REPORT\n"
        f"Voltage: {voltage_v:.2f} V\n"
        f"Resistance: {resistance_ohm:.2f} ohm\n"
        f"Current: {current_a:.3f} A\n"
        f"Power: {power_w:.3f} W\n"
        f"Status: {status}"
    )

# Store one component case.
voltage_v = 12.0
resistance_ohm = 100.0
rating_w = 2.0

# Execute the calculation and decision chain.
current_a = calculate_current(voltage_v, resistance_ohm)
power_w = calculate_power(voltage_v, current_a)
status = classify_power(power_w, rating_w)
report = format_resistor_report(
    voltage_v, resistance_ohm, current_a, power_w, status
)

# Display the report.
print(report)
```

Code 4.14: Decomposed resistor evaluation.

This evaluation does not prove that the resistor is appropriate under every thermal, transient, tolerance, or environmental condition. It applies only the stated power-rating comparison.

4.8.2 Practice 11: identify interfaces and data flow

For the resistor program above, draw or describe the data flow.

The data flow is:

- voltage and resistance enter `calculate_current`;
- current and voltage enter `calculate_power`;
- power and rating enter `classify_power`;
- the selected inputs and returned results enter `format_resistor_report`.

Each output must match the meaning and unit expected by the next function.

4.8.3 Practice 12: refactor repeated conversions

A script converts three temperatures from Celsius to kelvin using repeated expressions. Refactor it.

```python
# Convert one Celsius temperature to kelvin.
def celsius_to_kelvin(temp_c):
    # Use the scale offset between Celsius and kelvin.
    return temp_c + 273.15

# Reuse the same tested conversion for several values.
temp_1_k = celsius_to_kelvin(0.0)
temp_2_k = celsius_to_kelvin(25.0)
temp_3_k = celsius_to_kelvin(100.0)

# Display the three results.
print(temp_1_k, temp_2_k, temp_3_k)
```

Code 4.15: Reusable Celsius-to-kelvin function.

4.9 Integrated practice: function-based bar design

Develop a complete, documented program that:

1. accepts force in newtons, area in square millimetres, and material strength in megapascals;
2. validates positive inputs;
3. converts units;
4. calculates stress and factor of safety;
5. applies a stated required factor of safety;
6. returns numerical results from calculation functions;
7. formats a clear report separately;
8. includes component tests.

```python
# Convert square millimetres to square metres.
def convert_mm2_to_m2(area_mm2):
    """Return area in m^2 from a positive area in mm^2."""
    # Apply the squared unit conversion.
    return area_mm2 / 1_000_000

# Convert megapascals to pascals.
def convert_mpa_to_pa(value_mpa):
    """Return pressure or stress in Pa from MPa."""
    # One MPa equals one million Pa.
    return value_mpa * 1_000_000

# Calculate uniform axial stress.
def calculate_stress(force_n, area_m2):
    """Return uniform axial stress in Pa."""
    # Apply sigma = F/A.
    return force_n / area_m2

# Calculate factor of safety.
def calculate_fos(strength_pa, stress_pa):
    """Return dimensionless strength-to-stress ratio."""
    # Use matching pressure units.
    return strength_pa / stress_pa

# Apply the stated factor-of-safety requirement.
def classify_design(fos, required_fos=2.0):
    """Return a status string for the stated instructional rule."""
    # Include equality in the passing category.
    if fos >= required_fos:
        return "ACCEPTABLE BY STATED RULE"
    return "NOT ACCEPTABLE BY STATED RULE"

# Coordinate the numerical workflow.
def evaluate_bar(force_n, area_mm2, strength_mpa,
                 required_fos=2.0):
    """
    Return stress in Pa, factor of safety, and status.

    Assumptions: inputs are positive; axial stress is uniform.
    """
    # Reject physically invalid values before division.
    if force_n <= 0 or area_mm2 <= 0 or strength_mpa <= 0:
        return None, None, "INVALID INPUT"

    # Convert the input units.
    area_m2 = convert_mm2_to_m2(area_mm2)
    strength_pa = convert_mpa_to_pa(strength_mpa)

    # Calculate the engineering quantities.
    stress_pa = calculate_stress(force_n, area_m2)
    fos = calculate_fos(strength_pa, stress_pa)

    # Apply the documented decision rule.
    status = classify_design(fos, required_fos)
```

```python

    # Return the results for reuse and presentation.
    return stress_pa, fos, status

# Format the results without recalculating them.
def format_bar_report(force_n, area_mm2, stress_pa, fos, status):
    """Return a multi-line bar-evaluation report."""
    # Handle the invalid-input result honestly.
    if stress_pa is None:
        return f"BAR DESIGN REPORT\nStatus: {status}"

    # Convert stress to MPa for a concise report.
    stress_mpa = stress_pa / 1_000_000

    # Return labeled results with units.
    return (
        "BAR DESIGN REPORT\n"
        f"Force: {force_n:.1f} N\n"
        f"Area: {area_mm2:.1f} mm^2\n"
        f"Stress: {stress_mpa:.2f} MPa\n"
        f"Factor of safety: {fos:.2f}\n"
        f"Status: {status}"
    )

# Component tests using independently known values.
assert convert_mm2_to_m2(1_000_000) == 1
assert convert_mpa_to_pa(1) == 1_000_000
assert calculate_stress(10, 2) == 5
assert calculate_fos(100, 20) == 5
assert classify_design(2.0, 2.0) == "ACCEPTABLE BY STATED RULE"

# Evaluate one representative component.
force_n = 10_000.0
area_mm2 = 500.0
strength_mpa = 250.0

stress_pa, fos, status = evaluate_bar(
    force_n=force_n,
    area_mm2=area_mm2,
    strength_mpa=strength_mpa,
    required_fos=2.0
)

# Build and display the final report.
report = format_bar_report(
    force_n, area_mm2, stress_pa, fos, status
)
print(report)
```

Code 4.16: Complete annotated bar-design program.

4.10 Readiness checklist for Week 5

Before moving to tuples and lists, you should be able to answer yes to the following:

- I can define and call a function without confusing the two events.
- I can identify parameters and arguments.
- I can explain positional, keyword, and default arguments.
- I can use `return` when a value must be reused.
- I understand why a function may return `None`.
- I can explain local scope using a simple call trace.
- I can replace a hidden global dependency with an explicit parameter.
- I can write a short engineering docstring.
- I can state caller assumptions and a function guarantee.
- I can test a known case and an important boundary.
- I can decompose a larger task into focused functions.
- I can describe the data flow among those functions.
- I can separate calculation from input and reporting.
- I can explain abstraction without saying that internal details are unimportant; they are hidden only after being implemented and tested responsibly.

Key idea

A good function is a small engineering promise. Its name states the job. Its parameters expose the required information. Its return value carries the result. Its tests and documentation justify trust.

Sources and Suggested Reading

These notes are organized around the Week 4 lecture-slide set and its stated source basis.

1. Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 4.
2. Yanwei Wang, *Programming for Engineers with Python*, working draft, August 2026, Chapter 7: functions, decomposition, abstraction, documentation, and testing.
3. John V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Chapter 4, Sections 4.1–4.2.

Recommended preparation and review

For each short function:

1. predict the returned value before execution;
2. trace argument-to-parameter binding;
3. distinguish local names from caller-side names;
4. explain the function's job in one sentence;
5. record input meaning and units;
6. state the returned quantity and unit;
7. identify assumptions and invalid cases;
8. run at least one hand-checkable test;
9. test an important boundary;
10. modify one call or requirement and explain what changes.

Scope of these notes

The material intentionally concentrates on the Week 4 scope: function definitions and calls, parameters, arguments, returned values, local scope, documentation, elementary testing, decomposition, and abstraction. Tuples and lists are developed in Week 5; recursion and dictionaries in Week 6; systematic testing, exceptions, and assertions in Week 7; and object-oriented programming in Weeks 8–9.

The authoritative Assignment 2 task, deadline, permitted resources, submission format, marking criteria, and AI-use rules are provided through Moodle. These reading notes support conceptual preparation but do not replace the official assignment instructions.

A good function is a small engineering promise.

Its name states the job. Its parameters expose the required information.
Its return value carries the result. Its tests and documentation justify trust.